Generative Adversarial Networks (GANs)

1. Fundamental Architecture of a GAN

A **Generative Adversarial Network (GAN)** is a generative deep learning model consisting of two neural networks that compete against each other:

- **Generator (G):** Creates synthetic data that attempts to resemble real data.
- **Discriminator (D):** Determines whether a given sample is real or generated by the generator.

Generator

The generator takes a random noise vector z as input and transforms it into a synthetic sample:

$$z \quad G(z)$$

For example, in an image-generation task, the generator receives random noise and produces an artificial image.

The generator's objective is to create samples realistic enough to **fool the discriminator into classifying them as real**.

Discriminator

The discriminator acts as a binary classifier. It receives either:

- A **real sample** from the training dataset, or
- A **fake sample** created by the generator.

Its output represents how likely the input is to be real.

Ideally:

- $D(x) \quad 1$ for real samples.
- $D(G(z)) \quad 0$ for generated samples.

Therefore, while the generator tries to create convincing fake samples, the discriminator tries to become better at detecting them.

How the Generator and Discriminator Interact

Training takes place in an alternating adversarial process:

1. **Train the discriminator**
    - Real samples are given to the discriminator and labeled as real.
    - The generator creates fake samples.
    - Fake samples are given to the discriminator and labeled as fake.
    - The discriminator updates its parameters to improve its ability to distinguish real from fake data.
2. **Train the generator**
    - Random noise is passed through the generator.
    - The generated samples are passed to the discriminator.
    - The generator receives feedback based on whether the discriminator detected its samples as fake.
    - The generator updates its parameters to become better at fooling the discriminator.

The process can be summarized as:

`Random Noise → Generator → Fake Sample → Discriminator → Real/Fake Prediction`

The two networks therefore have competing objectives:

> ***Generator:*** *"Create samples that the discriminator believes are real."*

> ***Discriminator:*** *"Correctly distinguish real samples from generated samples."*

Theoretical Goal

The theoretical objective of GAN training is for the generator to learn the probability distribution of the real training data.

The original GAN objective is:

```
min_G max_D V(D,G) = E[log D(x)] + E[log(1 - D(G(z)))]
```

The discriminator attempts to maximize this objective, while the generator attempts to minimize it.

Ideally, training reaches a point where:

```
p_generator = p_data
```

At this point, the generated samples follow the same distribution as the real samples.

The discriminator can no longer reliably distinguish between real and generated data. Therefore, its prediction becomes approximately:

`D(x) = 0.5`

This represents the theoretical equilibrium of the adversarial process.

---

2. Mode Collapse

What Is Mode Collapse?

Mode collapse occurs when a GAN's generator produces only a limited variety of outputs instead of representing the full diversity of the training data.

For example, suppose a GAN is trained using images of handwritten digits from `0` through `9`.

Ideally, it should generate examples of all digits:

`0, 1, 2, 3, 4, 5, 6, 7, 8, 9`

However, suppose the generator discovers that images resembling `3` and `8` are particularly effective at fooling the discriminator.

Instead of learning all ten digits, it may repeatedly generate variations of only `3` and `8`.

The generated images might individually look realistic, but the generator has failed to represent the complete diversity of the dataset.

This phenomenon is called **mode collapse**.

Why Does Mode Collapse Occur?

The generator's primary objective is to **fool the discriminator**, not explicitly to produce diverse samples.

If the generator discovers one particular type of output that successfully fools the discriminator, producing similar outputs repeatedly may temporarily reduce its loss.

This can create the following behavior:

1. The generator discovers an output that successfully fools the discriminator.

2. It begins generating many similar samples.
3. The discriminator eventually learns to recognize these samples.
4. The generator moves toward another type of successful output.
5. This process may repeat without the generator learning the complete data distribution.

As a result, some modes of the real data distribution may never be represented by the generator.

Technique 1: Minibatch Discrimination

Minibatch discrimination allows the discriminator to consider relationships between multiple samples in the same batch instead of evaluating every sample completely independently.

Suppose the generator produces:

```
Sample 1: Cat A
Sample 2: Cat A
Sample 3: Cat A
Sample 4: Cat A
Sample 5: Cat A
```

Each image may individually look realistic. However, the entire batch has very little diversity.

Minibatch discrimination allows the discriminator to recognize this similarity between generated samples.

As a result, the generator is encouraged to produce something more diverse:

```
Sample 1: Cat A
Sample 2: Cat B
Sample 3: Cat C
Sample 4: Cat D
Sample 5: Cat E
```

How it helps: The discriminator can penalize batches containing highly repetitive samples. This encourages the generator to create a wider variety of outputs and therefore reduces mode collapse.

Technique 2: Wasserstein GAN (WGAN)

A **Wasserstein GAN (WGAN)** modifies the original GAN training objective by using the **Wasserstein distance**, also known as the **Earth Mover's Distance**.

Instead of using a traditional discriminator that produces a probability between `0` and `1`, WGAN uses a **critic** that produces a real-valued score.

The objective is conceptually:

`Real Data Score - Generated Data Score`

One major problem with traditional GANs is that the discriminator can sometimes become too strong. When this happens, the generator may receive weak or uninformative gradients, making training unstable.

WGAN provides smoother and more meaningful gradients to the generator.

How it helps:

- Provides more useful gradients to the generator.
- Makes GAN training more stable.
- Reduces problems caused by vanishing gradients.
- Improves convergence.
- Reduces the likelihood of mode collapse.

An improved version, **WGAN-GP**, introduces a **gradient penalty** to enforce the required Lipschitz constraint more effectively than the weight clipping used in the original WGAN.

---

3. Significant GAN Variants

Three important extensions of the original GAN architecture are:

1. **DCGAN (Deep Convolutional GAN)**
2. **Conditional GAN (CGAN)**
3. **CycleGAN**

3.1 DCGAN — Deep Convolutional GAN

Problem Addressed

The original GAN framework does not define an architecture specifically designed for image generation.

Basic fully connected neural networks are not ideal for images because they do not naturally take advantage of the spatial relationships between neighboring pixels.

Main Innovation

DCGAN combines GANs with **Convolutional Neural Networks (CNNs)**.

The generator uses learned upsampling operations, commonly transposed convolutions, to transform a random noise vector into an image.

A simplified generator architecture is:

`Random Noise` `Reshape` `Transposed Convolutions` `Generated Image`

The discriminator performs approximately the opposite process:

`Image` `Convolutional Layers` `Feature Extraction` `Real/Fake Prediction`

Important architectural ideas associated with DCGAN include:

- Convolutional layers instead of fully connected hidden layers.
- Strided convolutions for downsampling.
- Transposed convolutions for learned upsampling.
- Batch normalization for more stable training.
- ReLU activations in much of the generator.
- LeakyReLU activations in the discriminator.
- `tanh` activation for the generator's output layer.

Benefit

DCGAN significantly improved GAN performance on image-generation tasks.

It allowed GANs to learn meaningful spatial features and produce higher-quality images, while also providing architectural guidelines that influenced many later GAN models.

3.2 Conditional GAN (CGAN)

Problem Addressed

A standard GAN provides little direct control over **what type of sample is generated**.

For example, consider a GAN trained on handwritten digits.

A standard GAN receives random noise:

`Random Noise` `Generator` `Random Digit`

The user cannot directly request:

> *"Generate the digit 7."*

Main Innovation

A **Conditional GAN (CGAN)** solves this problem by providing additional information, called a **condition**, to both the generator and discriminator.

The condition can include:

- Class labels
- Text descriptions
- Image attributes
- Other images
- Semantic information

The generator receives:

`Generator(random noise, condition)`

The discriminator receives:

`Discriminator(image, condition)`

For example, suppose the condition is:

`Class Label = 7`

The process becomes:

`Random Noise + Label 7  Generator  Image of Digit 7`

The discriminator then determines two things:

1. Does the generated image look realistic?
2. Does the image correctly match the specified condition?

Benefit

Conditional GANs provide **control over the generation process**.

They can be used for:

- Class-specific image generation.
- Text-to-image generation.
- Attribute-controlled image generation.
- Image-to-image translation.

- Super-resolution.

The major improvement over a standard GAN is therefore the ability to generate samples based on specific user-defined conditions.

3.3 CycleGAN

Problem Addressed

Many image-to-image translation tasks involve converting an image from one domain into another.

Examples include:

- Horse Zebra
- Summer Winter
- Photograph Painting
- Painting Photograph

Traditional supervised image-to-image translation usually requires **paired training data**.

For example:

```
Input:  A specific horse image
Target: The exact same scene transformed into a zebra image
```

Obtaining thousands of perfectly paired images can be difficult or impossible.

Main Innovation

CycleGAN allows image-to-image translation using **unpaired datasets**.

For example, we only need:

```
Dataset X = Collection of horse images
Dataset Y = Collection of zebra images
```

The horse images do not need to correspond to particular zebra images.

CycleGAN uses:

- **Two generators**
- **Two discriminators**

The first generator performs:

`Generator G: Horse  Zebra`

The second performs:

`Generator F: Zebra  Horse`

Each domain has its own discriminator to determine whether the translated images look realistic.

Cycle-Consistency Loss

The key innovation of CycleGAN is **cycle consistency**.

Suppose we begin with a horse image:

`Horse  Generator G  Zebra`

The generated zebra image is then translated back:

`Generated Zebra  Generator F  Reconstructed Horse`

Ideally:

`Original Horse ≈ Reconstructed Horse`

Similarly:

`Original Zebra ≈ Reconstructed Zebra`

This can be represented as:

$F(G(x)) \approx x$

and:

$G(F(y)) \approx y$

This requirement is enforced using **cycle-consistency loss**.

Why Cycle Consistency Is Important

Without cycle consistency, a generator could take a horse image and produce any realistic zebra image without preserving important information from the original horse image.

Cycle consistency forces the transformation to preserve important structural and content information.

Benefit

CycleGAN enables image-to-image translation **without requiring paired training examples**.

Common applications include:

- Horse-to-zebra transformation.
- Summer-to-winter transformation.
- Photo-to-painting conversion.
- Artistic style transfer.
- Domain adaptation.

Summary

A **Generative Adversarial Network** consists of two competing neural networks: a **generator** and a **discriminator**. The generator attempts to create realistic synthetic data, while the discriminator attempts to distinguish generated samples from real samples. The theoretical goal is for the generator to learn the real data distribution so accurately that the discriminator can no longer reliably distinguish real from generated data.

Mode collapse is an important GAN training problem in which the generator produces only a limited variety of samples. **Minibatch discrimination** reduces this problem by allowing the discriminator to detect repetitive outputs, while **Wasserstein GANs** improve training stability by providing smoother and more meaningful gradients.

The original GAN architecture has also inspired several important variants:

GAN Variant	Problem Addressed	Main Innovation
DCGAN	Standard GANs are not specifically optimized for images	Uses convolutional and transposed convolutional architectures
Conditional GAN	Standard GANs provide little control over generated content	Conditions the generator and discriminator on labels or other information
CycleGAN	Image translation normally requires paired training data	Uses two generators, two discriminators, and cycle-consistency loss for unpaired translation

These developments have made GANs more stable, controllable, and effective for a wide variety of image-generation and image-translation tasks.